

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №1
по дисциплине «Представление знаний в системах искусственного
интеллекта»

Тема: Изучение основных возможностей и базовых команд среды
CLIPS. Разработка демонстрационной экспертной системы.

Студент гр. 1306

Шаврин А.П.

Преподаватель

Сучков А.И.

Санкт-Петербург

2025

Цель работы.

Изучить основные возможности и базовые команды среды CLIPS.
Разработать демонстрационную экспертную систему.

Основные теоретические положения.

Среда CLIPS (C Language Integrated Production System) предназначена для построения экспертных систем (ЭС). Она поддерживает три основных способа представления знаний:

- продукционные правила для представления эвристических, основанных на опыте знаний;
- функции для представления процедурных знаний;
- объектно-ориентированное программирование.

Для сброса среды CLIPS в исходное состояние используется команда (clear) или соответствующий пункт меню Execution.

В CLIPS поддерживаются восемь простейших типов данных: целые числа (integer), числа с плавающей запятой (float), символьный (symbol), строковый (string), внешний адрес (external-address), адрес факта (fact-address), имя экземпляра (instance-name) и адрес экземпляра (instance-address).

Факты являются одной из основных форм представления информации в CLIPS-системах и используются правилами для вывода новых фактов из имеющихся. Все текущие факты в CLIPS помещаются в список фактов (fact-list).

По формату представления в CLIPS выделяют два типа фактов: упорядоченные и неупорядоченные. Упорядоченный факт состоит из заключенной в скобки последовательности одного или более полей, разделенных пробелами. Поля в неупорядоченном факте могут быть любыми простейшими типами данных (за исключением первого поля, которое должно быть символьного типа). Первое поле упорядоченного факта специфицирует отношение, которое применяется к остальным полям факта.

Для работы с фактами используются следующие команды: `assert` – добавляет факт в факт-список; `retract` – удаляет факт из списка; `modify` – модифицирует список; `duplicate` – дублирует факт.

Некоторые команды, такие как `retract`, `modify` и `duplicate`, требуют, чтобы факты были идентифицированы. Для этой цели используется либо индекс факта (`fact-index`), либо адрес факта (`fact-address`). Индекс факта - уникальный целочисленный индекс, приписываемый факту всякий раз, когда факт добавляется (или модифицируется). Индекс факты начинаются с нуля и инкрементируются при каждом новом или измененном факте. Идентификатор факта (`fact identifier`) представляет собой краткую нотацию для отображения факта. Он состоит из символа “f”, за которым через тире следует индекс факта.

Для задания исходного множества фактов используется конструкция `deffacts`, со следующим синтаксисом: (`deffacts` <имя_группы_фактов> ["<комментарий>"] <факт>), где <имя_группы_фактов> - идентификатор символьного типа; <комментарий> - необязательное поле комментария; <факт>) - произвольная последовательность фактов, записанных через разделитель.

Факты, определенные конструкцией `deffacts` добавляются в список фактов всякий раз при выполнении команды `reset`. Для задания правил используется конструкция `defrule` со следующим синтаксисом: (`defrule` <имя_правила> ["<комментарий>"] [<объявление>] ; <условный элемент>* ; Левая часть правила (антецедент) => <действие>) ; Правая часть правила (консеквент) где <имя_правила> - идентификатор символьного типа, уникальный для данной группы правил; <комментарий> - необязательное поле комментария; <условный элемент> - произвольная последовательность условных элементов; <действие>* - произвольная последовательность действий.

Постановка задачи.

1. Изучение базовых команд и конструкций CLIPS.

1.1. Запустить систему CLIPS (файл `clipswin.exe`). Активизировать окно просмотра текущего списка фактов (подпункт “Facts Window” пункта

“Windows” главного меню). Выполнить следующую последовательность действий, фиксируя после каждого шага состояние списка фактов:

1. сбросить систему в исходное состояние командой (clear);
2. выполнить начальную установку командой (reset) или комбинацией клавиш Ctrl-E;
3. ввести 3 любых упорядоченных факта командой (assert), например:
 - a. CLIPS> (assert (n n) (m m) (p p));
4. повторно выполнить сброс командой (reset);
5. установить 3 ранее вводимых упорядоченных факта в качестве исходных фактов, используя конструкцию (deffacts);
6. выполнить сброс командой (reset).

1.2. Активизировать дополнительно окно просмотра агенды (подпункт “Agenda Window” пункта “Windows” главного меню). Выполнить следующую последовательность действий, фиксируя после каждого шага состояния списка фактов и агенды:

- используя конструкцию (defrule), ввести три правила, такие, что антецеденты первых двух правил сопоставляются с комбинацией фактов, заданных ранее конструкцией (deffacts), а консеквенты этих правил добавляют новые факты, сопоставляемые с антецедентом третьего правила. Пусть, например, X, Y и Z – факты, заданные конструкцией (deffacts). Тогда структура вводимых правил может быть представлена следующим образом:

$X \& Y \Rightarrow V;$

$Y \& Z \Rightarrow W;$

$V \& W \Rightarrow U;$

- выполнить по шагам активизацию правил (используя «горячую» комбинацию Ctrl-T).

2. Разработка демонстрационной экспертной системы.

2.1. Выбрать (придумать) предметную область демонстрационной экспертной системы (ЭС). Сформировать, пользуясь редактором CLIPS, базу

знаний демонстрационной ЭС для выбранной предметной области и сохранить ее в файле rulebase.clp.

В качестве примера использовать фрагмент ЭС, которая вырабатывает рекомендации студенту накануне зачета и имеет четыре входные переменные («число дней до зачета», «количество несделанных лабораторных работ (в %)», «температура на улице» и «наличие осадков»), две промежуточные («свободное время» и «погода») и выходную переменную («рекомендуемые действия»). Данный фрагмент базы знаний (БЗ) содержится в прилагаемом файле rulebase.clp. Диаграмма зависимости переменных показана на рисунке 1, в скобках указаны возможные имена переменных.

Общее количество правил в БЗ должно быть не менее 25. Количество значений переменных должно выбираться таким образом, чтобы БЗ отвечала требованию полноты, т.е. содержала правила, соответствующие любым сочетаниям значений переменных в левых частях правил. В примере, если переменная «свободное время» имеет 3 значения («отсутствует», «мало» и «много»), а переменная «погода» – 2 значения («плохая» и «хорошая»), то максимальное число правил для определения переменной «действие» будет равно 6.

Наивысшая оценка за работу может быть получена при условии, что ЭС задает пользователю не сразу все вопросы о недостающих значениях переменных и только потом проводит экспертизу, а по мере их необходимости, обусловленной процессом экспертизы с учетом ответов на предыдущие вопросы, отраженном на диаграмме зависимости переменных (в примере это требование не соблюдено).

2.2. Для активизации ЭС в среде CLIPS использовать пакетный файл Run_Lab_1.bat, который может быть запущен с использованием пункта «Load Batch» меню «File».

2.3. Оттестировать ЭС на различных комбинациях входных значений в пошаговом режиме. Представить отчет о практической работе с приложением исходного текста ЭС.

Роли:

Гранкин Захар – выполнение пункта 1, разработка структуры ЭС, написание правил для ввода данных пользователя, написание промежуточных правил ЭС, включая правила получения срочности.

Шаврин Алексей – тестирование пункта 1, разработка структуры ЭС, написание правил для получения рекомендаций ЭС, составление отчета.

Новицкий Михаил – разработка структуры ЭС, написание правила для валидации опроса ЭС, тестирование ЭС и предоставление скриншотов.

Выполнение работы.

1. Изучение базовых команд и конструкций CLIPS.

1.1. Была установлена и запущена программа CLIPS. Были открыты рядом 2 окна: окно просмотра текущего списка фактов и диалоговое окно, представлено на рис. 1.



Рисунок 1 – Диалоговое окно и окно фактов программы CLIPS

1.1.1. Выполнен сброс программы в исходное состояние командой (clear) и начальная установка командой (reset). Представлено на рис. 2 и 3.



Рисунок 2 – Сброс программы – команда clear



Рисунок 3 – Начальная установка программы – команда reset

1.1.2. Введены 3 упорядоченные факта в качестве входных данных командой (assert), список фактов пополнился, результат на рис. 4.

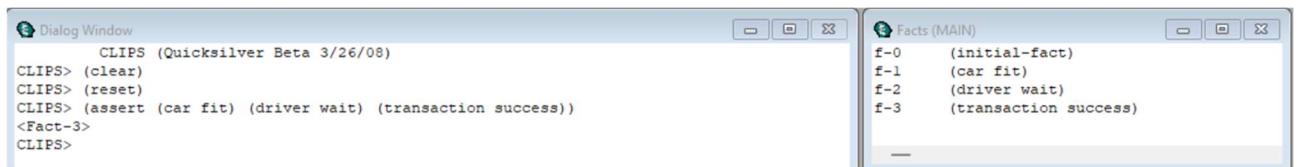


Рисунок 4 – Добавление фактов – команда assert

1.1.3. Программа была повторно сброшена командой (reset). Список фактов очистился, представлено на рис. 5.

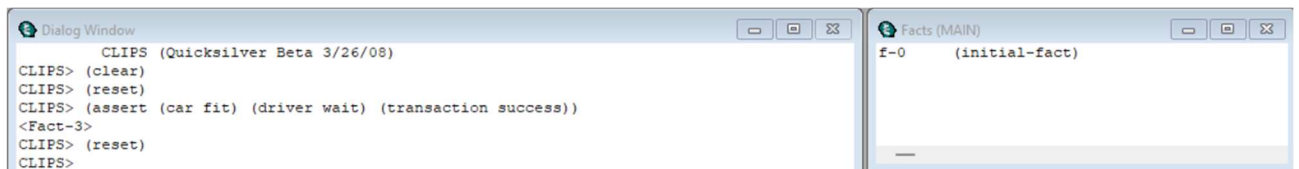


Рисунок 5 – Сброс к начальном состоянию программы – команда reset

1.1.4. Установлены 3 ранее вводимых факта в качестве исходных фактов с помощью конструкции (deffacts), представлено на рис. 6

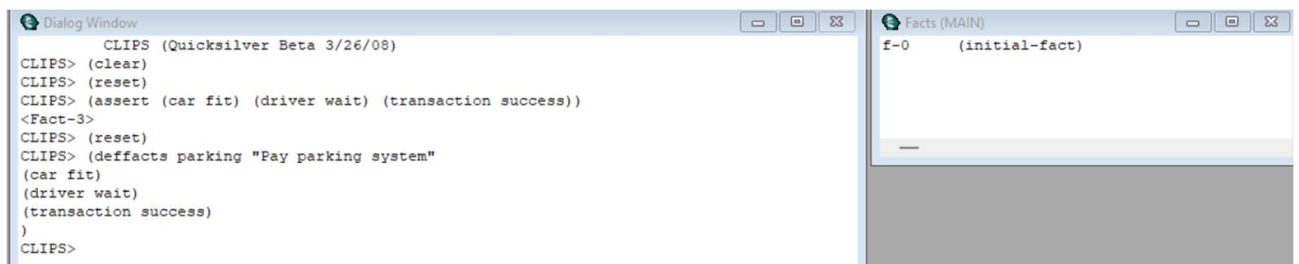


Рисунок 6 – Установка трех фактов в качестве исходных с конструкцией (deffacts).

1.1.5 Программа была установлена в начальное состояние командой (reset). список фактов пополнился фактами определенными через deffacts, представлено на рис. 7.

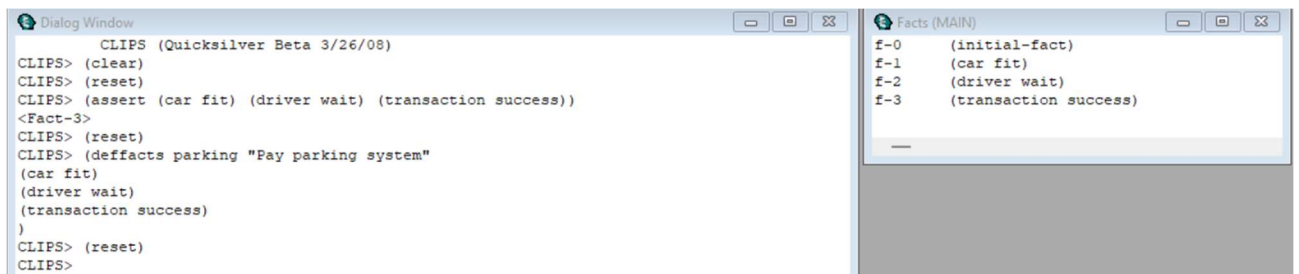


Рисунок 7 – Начальная установка программы – команда reset

1.2 Были введены 3 правила R1, R2 и R3 с помощью конструкции (defrule). Первые 2 правила добавятся в agenda поскольку их антецедент выполняется, а третье правило добавится в agenda только после отработки первых 2х.

Учитывая стековую структуру агенды правила вводились в следующем порядке. Сначала добавлено правило R2 (driver wait) и (transition success) => (payment received), результат представлен на рисунке 8.

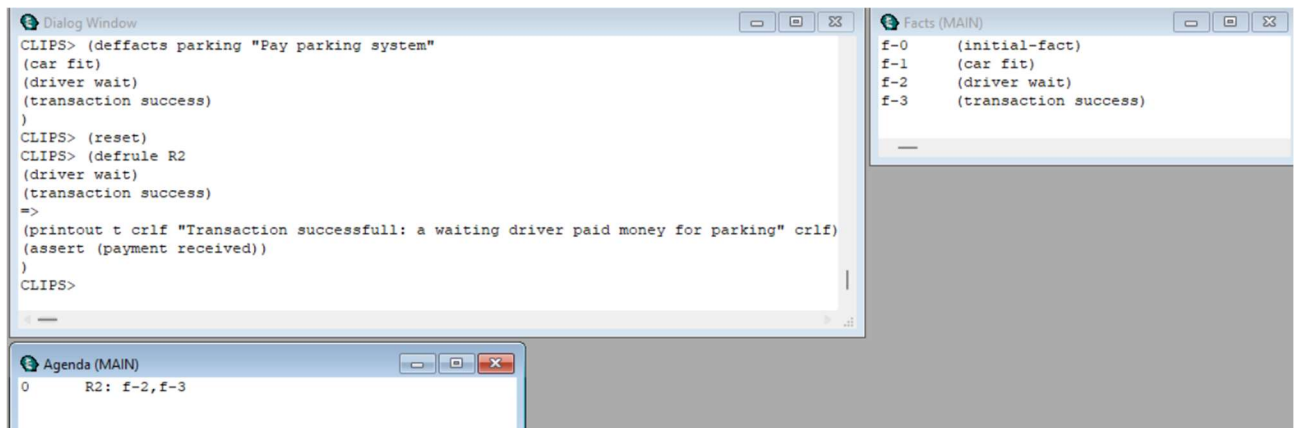


Рисунок 8 – Добавление правила R2 с помощью конструкции (defrule).

Потом добавлено правило R1 (driver wait) и (car fit) => (spot available), результат представлен на рисунке 9.

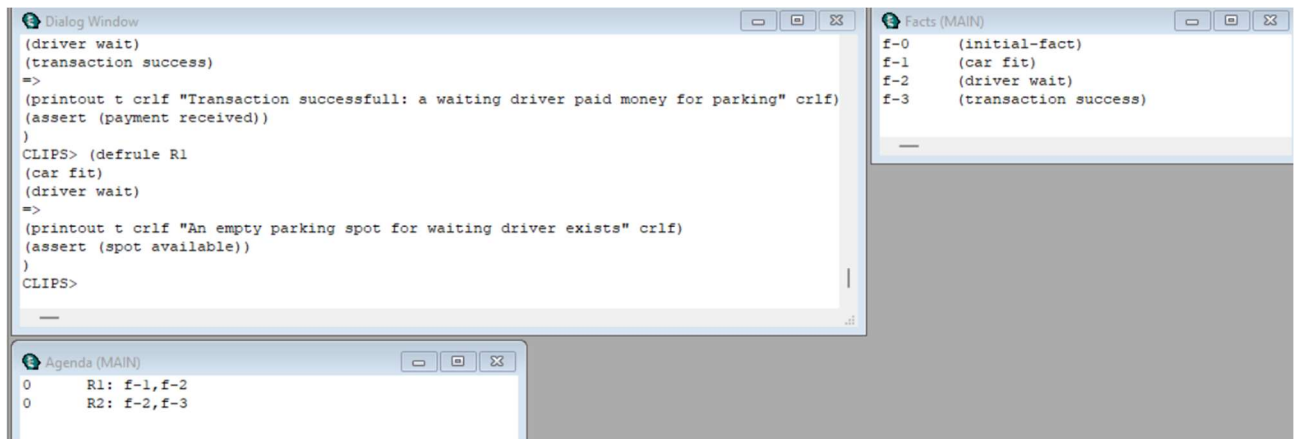


Рисунок 9 – Добавление правила R1 с помощью конструкции (defrule). Затем добавлено правило R3 (spot available) и (payment recieved) => (parking granted), правило не добавилось в agenda, результат на рисунке 10.

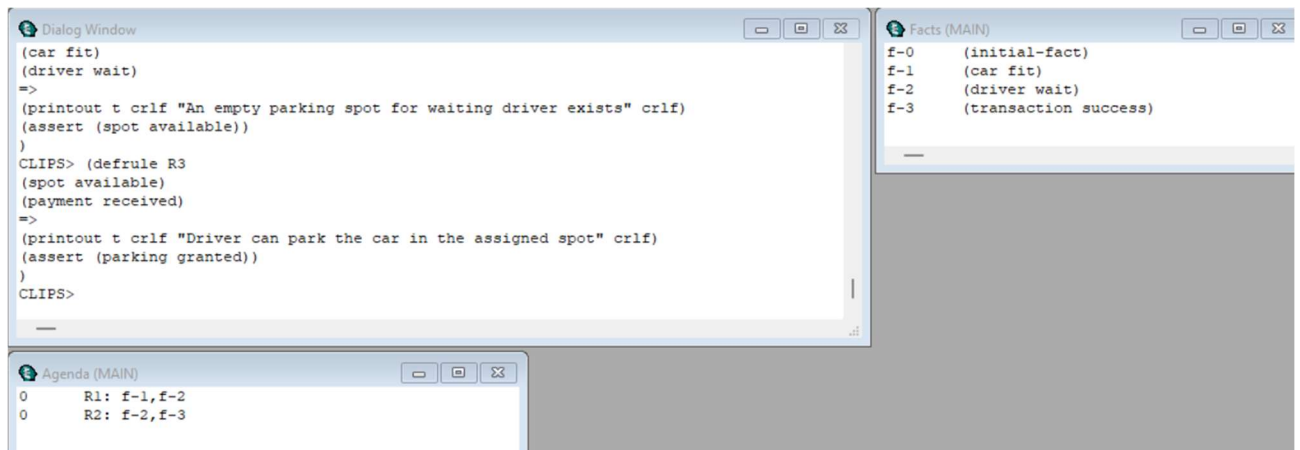


Рисунок 10 – Добавление правила R3 с помощью конструкции (defrule).

1.2.1 Выполним активизацию правил по шагам с помощью команды (run 1) (представлено на рис. 11 – рис. 13).

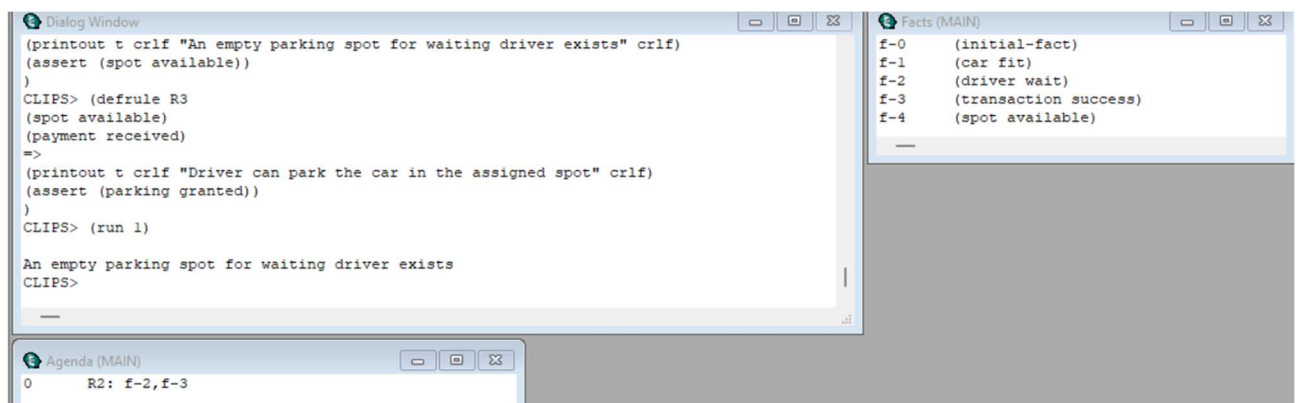


Рисунок 11 – Список фактов и агенда после выполнения первого шага.

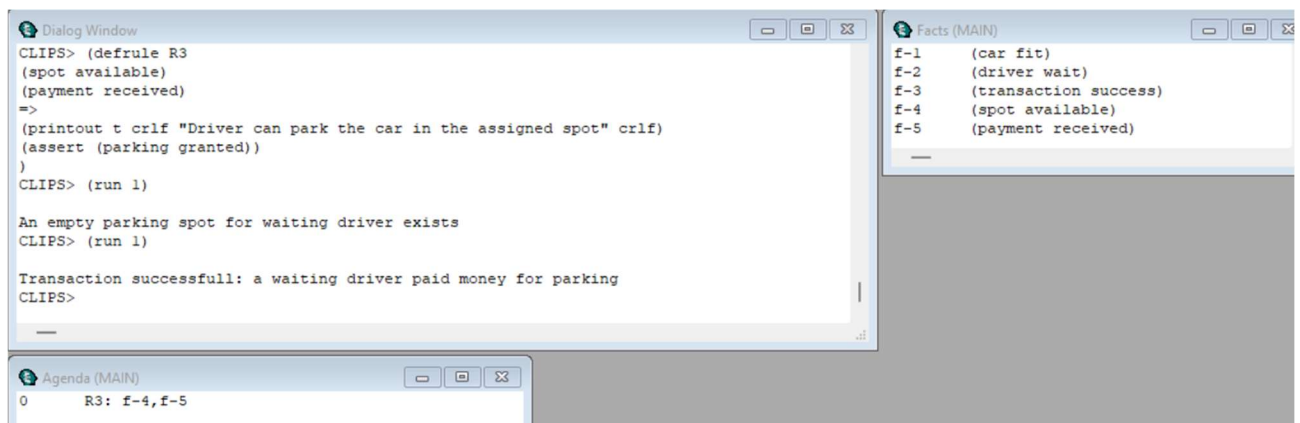


Рисунок 12 – Список фактов и агенда после выполнения второго шага.

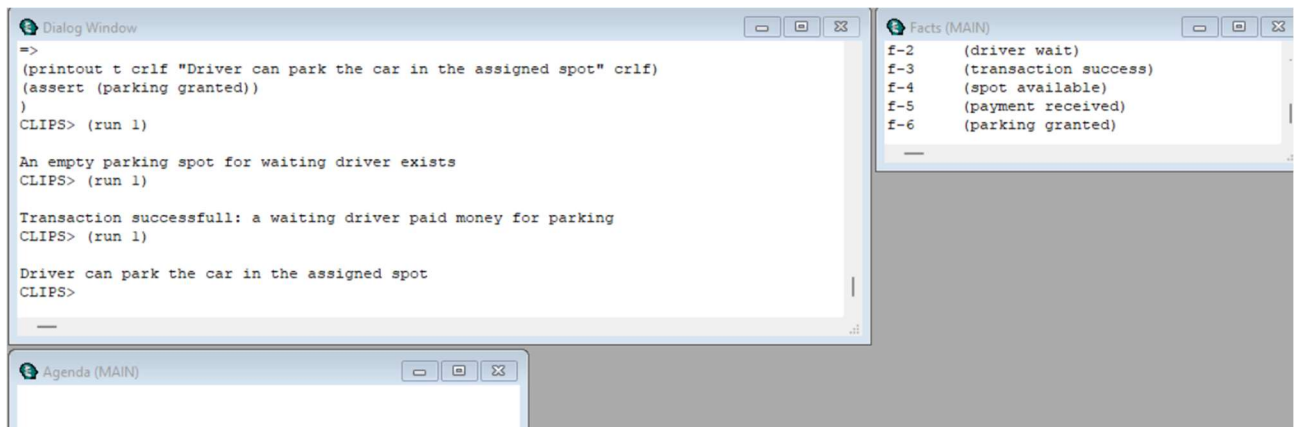


Рисунок 13 – Список фактов и агенда после выполнения третьего шага. Правило R1 – выполнилось первым, затем отработало правило R2. И только потом в agenda попало правило R3 и выполнившись добавило факт (parking granted).

2. Разработка экспертной системы.

Предметная область ЭС относится к банковскому обслуживанию, где клиент может столкнуться с различными типами проблем и различными сервисами банка. Входные параметры, которые вводит пользователь, их возможные значения и параметр, определяющий необходимо ли вводить пользователю значение, а также смысл данной переменной, представлены в таблице 1.

Таблица 1. Параметры ЭС.

Параметр	Возможные значения	Когда запрашивается	Значение
service-type	card / contribution / app	Всегда первым	Определяет сферу проблемы
problem-category	security-threat / technical-failure / provide-info / other	После выбора сервиса	Определяет тип проблемы
customer-urgency	yes / no	Только если необходимо	Показывает критичность ситуации

Диаграмма зависимостей переменных представлена на рисунке 14.

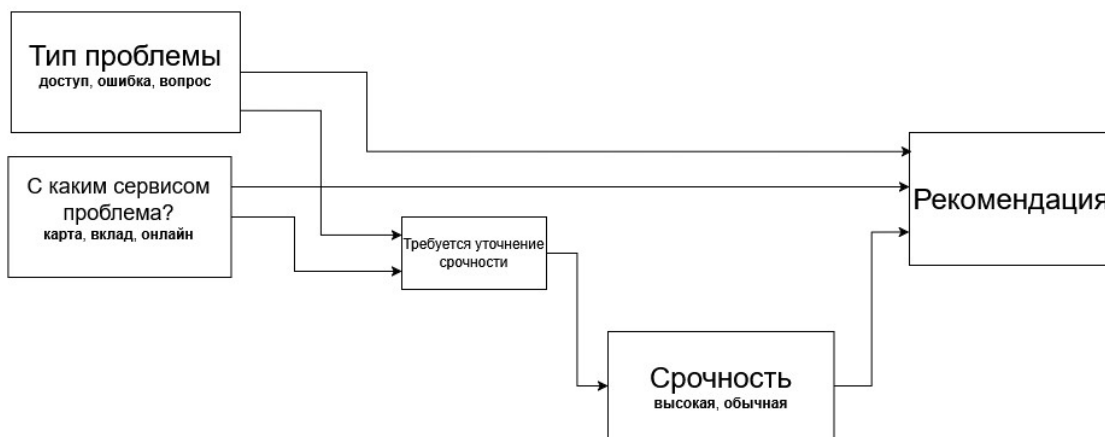


Рисунок 14 – Диаграмма зависимостей переменных.

Дерево возможных решений представлено на рисунке 15.

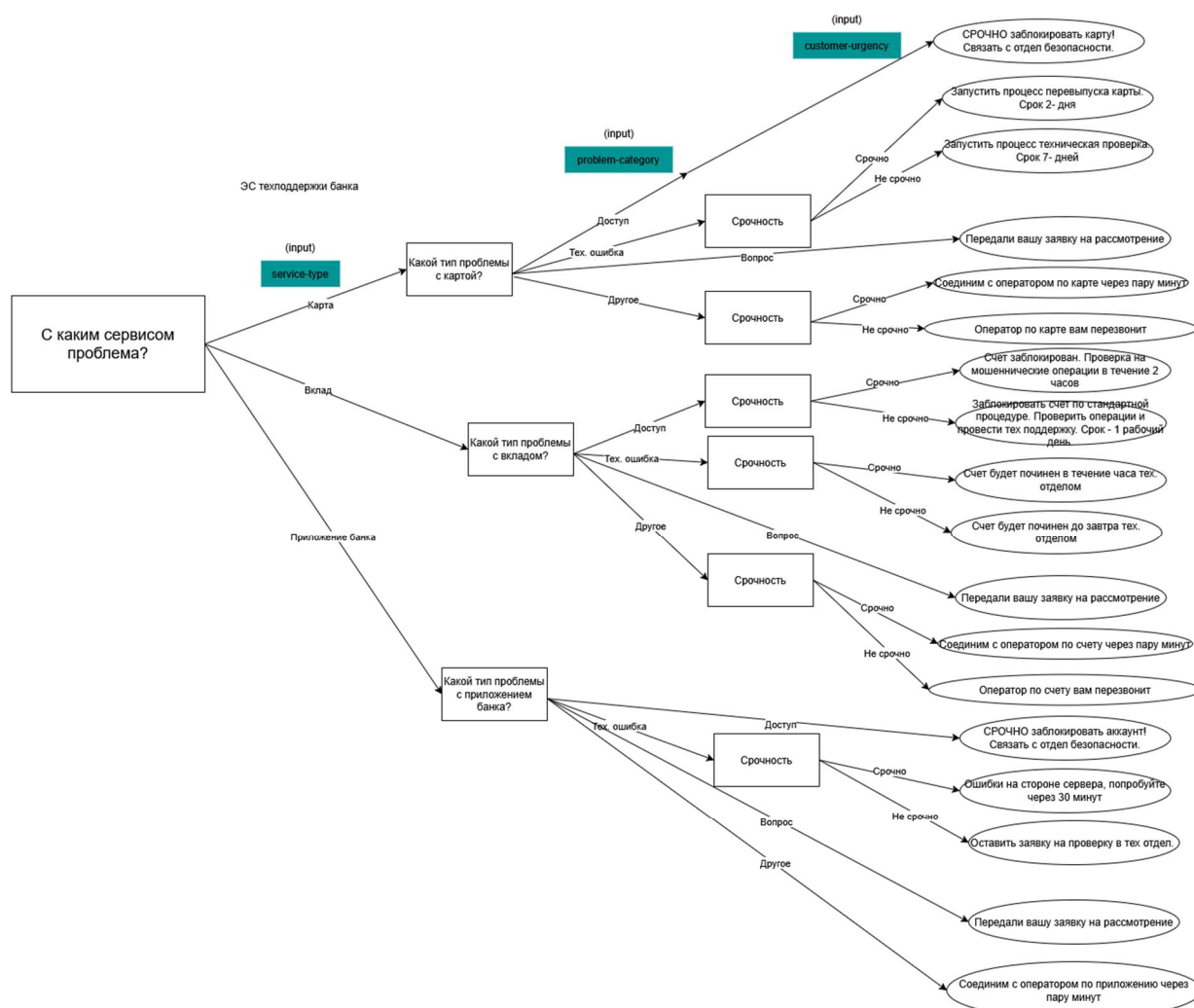


Рисунок 15 – Дерево решений.

Была разработана функция для валидации вводимых значений на принадлежность введенного пользователя варианту одному из возможных для данного вопроса. Функция представлена на рисунке 16.

```
(deffunction validate-input (?input-text ?valid-options)
  (while TRUE
    (printout t crlf ?input-text crlf)
    (bind ?input (read))
    (if (member$ ?input ?valid-options)
        then (return ?input)
        )
    )
  )
)
```

Рисунок 16 – Функция валидации ввода.

Были созданы правила для получения фактов от пользователя о типе сервиса и проблем с сервисом (рис 17).

```
(defrule get-service-type
  =>
  (bind ?service-type (validate-input
    "System supports only card, contribution or app choices
    Enter a service where you face the problem: "
    (create$ card contribution app)
  ))
  (assert (service-type ?service-type))
)

(defrule get-card-problem
  (service-type card)
  =>
  (bind ?problem-category (validate-input
    "System supports only security-threat, technical-failure or provide-info choices.
    If nothing suits your case, you may choose option other.
    What type of problem happened with your card?"
    (create$ security-threat technical-failure provide-info other)
  ))
  (assert (problem-category ?problem-category))
)

(defrule get-contribution-problem
  (service-type contribution)
  =>
  (bind ?problem-category (validate-input
    "System supports only security-threat, technical-failure or provide-info choices.
    If nothing suits your case, you may choose option other.
    What type of problem happened with your contribution?"
    (create$ security-threat technical-failure provide-info other)
  ))
  (assert (problem-category ?problem-category))
)

(defrule get-app-problem
  (service-type app)
  =>
  (bind ?problem-category (validate-input
    "System supports only security-threat, technical-failure or provide-info choices.
    If nothing suits your case, you may choose option other.
    What type of problem happened with your app?"
    (create$ security-threat technical-failure provide-info other)
  ))
  (assert (problem-category ?problem-category))
)
```

Рисунок 17 – Получение типа сервиса и проблем с сервисом.

Была написана функция установки дефолтного значения срочности (рис 18).

Это правило служит для того, чтобы установить значение срочности по

умолчанию — customer-urgency no, если пользователь не был обязан сам выбирать срочность. В нем используется declare (salience -100) для понижения приоритета правила. Это нужно для того, чтобы, когда пользователь выбирает service-type и problem-category и запускается дерево правил, которые могут задать срочность или не задавать, если никакое правило, задающее срочность, не сработало, то нужно применить значение по умолчанию. Но если бы правило "по умолчанию" имело такое же приоритетное значение, как остальные, оно могло бы сработать раньше, чем система успеет спросить у пользователя. Так правило не мешает другим правилам, ждёт, пока пользователь введёт срочность, если нужно, или правило срочности будет выведено другими правилами и только в самом конце, если срочность не установлена → устанавливает по

```
(defrule set-default-low-urgency-if-not-provided
  (declare (salience -100))
  (service-type ?)
  (problem-category ?)
  (not (customer-urgency ?))
  =>
  (assert (customer-urgency no))
)
```

Рисунок 18 – Правило задания дефолтной срочности.

Также были созданы правила для получения фактов от пользователя о срочности проблемы с конкретным сервисом (рис 19).


```

(defrule get-technical-failure-card-problem-urgency
  (service-type card)
  (problem-category technical-failure)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-non-trivial-card-problem-urgency
  (service-type card)
  (problem-category other)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-security-threat-contribution-problem-urgency
  (service-type contribution)
  (problem-category security-threat)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-technical-failure-contribution-problem-urgency
  (service-type contribution)
  (problem-category technical-failure)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-non-trivial-contribution-problem-urgency
  (service-type contribution)
  (problem-category other)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-technical-failure-app-problem-urgency
  (service-type app)
  (problem-category technical-failure)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

```

Рисунок 19 – Получение срочности для сервиса и проблемы с сервисом. После были заданы правила рекомендаций для каждой комбинации входных данных (рис 20 - 24).

```

(defrule output-card-security-threat-recommendation
  (service-type card)
  (problem-category security-threat)
  (or (customer-urgency yes)
    (customer-urgency no))
  =>
  (printout t crlf "Your card will be blocked. Info sent to security department" crlf)
  (assert (recommendation block-card))
  (assert (notify security-department))
)

(defrule output-card-urgent-technical-failure-recommendation
  (service-type card)
  (problem-category technical-failure)
  (customer-urgency yes)
  =>
  (printout t crlf "Your card will be re-released in two days" crlf)
  (assert (recommendation urgent-card-re-release))
  (assert (notify tech-department))
)

(defrule output-card-non-urgent-technical-failure-recommendation
  (service-type card)
  (problem-category technical-failure)
  (customer-urgency no)
  =>
  (printout t crlf "Your card will be re-released in seven days" crlf)
  (assert (recommendation non-urgent-card-re-release))
  (assert (notify tech-department))
)

```

Рисунок 20 – Предоставление рекомендации.

```

(defrule output-card-provide-info-recommendation
  (service-type card)
  (problem-category provide-info)
  (or (customer-urgency yes)
      (customer-urgency no))
  =>
  (printout t crlf "Application to receive information about your card is transmitted for consideration" crlf)
  (assert (recommendation transmit-card-info-application))
  (assert (notify tech-support))
)

(defrule output-card-non-trivial-urgent-problem-recommendation
  (service-type card)
  (problem-category other)
  (customer-urgency yes)
  =>
  (printout t crlf "You will be connected with our contact center in a few minutes to discuss a problem with your card" crlf)
  (assert (recommendation connect-with-contact-center-about-urgent-card-problem))
  (assert (notify contact-center))
)

(defrule output-card-non-trivial-non-urgent-problem-recommendation
  (service-type card)
  (problem-category other)
  (customer-urgency no)
  =>
  (printout t crlf "Our first available operator will recall you to discuss problem with your card" crlf)
  (assert (recommendation connect-with-contact-center-about-non-urgent-card-problem))
  (assert (notify contact-center))
)

(defrule output-urgent-contribution-security-threat-recommendation
  (service-type contribution)
  (problem-category security-threat)
  (customer-urgency yes)
  =>
  (printout t crlf "Your contribution is blocked. All recent operations will be inspected during the next few hours" crlf)
  (assert (recommendation block-contribution-with-fraud-inspection))
  (assert (notify security-department))
)

```

Рисунок 21 – Предоставление рекомендации.

```

(defrule output-non-urgent-contribution-security-threat-recommendation
  (service-type contribution)
  (problem-category security-threat)
  (customer-urgency no)
  =>
  (printout t crlf "Your contribution is blocked by standart procedure. All recent operations will be inspected during till tomorrow.
    Technical support will call you for the next hour" crlf)
  (assert (recommendation block-contribution-suspicious-transactions-inspection))
  (assert (notify security-department))
  (assert (notify tech-support))
)

(defrule output-urgent-contribution-technical-failure-recommendation
  (service-type contribution)
  (problem-category technical-failure)
  (customer-urgency yes)
  =>
  (printout t crlf "Your contribution will be fixed in an hour by tech department." crlf)
  (assert (recommendation transmit-critical-bug-to-tech-department))
  (assert (notify tech-department))
)

(defrule output-non-urgent-contribution-technical-failure-recommendation
  (service-type contribution)
  (problem-category technical-failure)
  (customer-urgency no)
  =>
  (printout t crlf "Your contribution will be fixed till tomorrow by tech department." crlf)
  (assert (recommendation transmit-average-bug-to-tech-department))
  (assert (notify tech-department))
)

(defrule output-contribution-provide-info-recommendation
  (service-type contribution)
  (problem-category provide-info)
  (or (customer-urgency yes)
      (customer-urgency no))
  =>
  (printout t crlf "Application to receive information about your contribution is transmitted for consideration" crlf)
  (assert (recommendation transmit-contribution-info-application))
)

(defrule output-contribution-non-trivial-urgent-problem-recommendation
  (service-type contribution)
  (problem-category other)
  (customer-urgency yes)
  =>
  (printout t crlf "You will be connected with our contact center in a few minutes to discuss a problem with your contribution" crlf)
  (assert (recommendation connect-with-contact-center-about-urgent-contribution-problem))
  (assert (notify contact-center))
)

```

Рисунок 22 – Предоставление рекомендации.

```

(defrule output-contribution-non-trivial-non-urgent-problem-recommendation
  (service-type contribution)
  (problem-category other)
  (customer-urgency no)
  =>
  (printout t crlf "Our first available operator will recall you to discuss problem with your contribution" crlf)
  (assert (recommendation connect-with-contact-center-about-non-urgent-contribution-problem))
  (assert (notify contact-center))
)

(defrule output-app-security-threat-recommendation
  (service-type app)
  (problem-category security-threat)
  (or (customer-urgency yes)
      (customer-urgency no))
  =>
  (printout t crlf "You account is temporarily blocked. Information sent to security department" crlf)
  (assert (recommendation block-account))
  (assert (notify security-department))
)

(defrule output-urgent-app-technical-failure-recommendation
  (service-type app)
  (problem-category technical-failure)
  (customer-urgency yes)
  =>
  (printout t crlf "Server-side problems. Retry your attempt in 30 minutes" crlf)
  (assert (recommendation report-about-server-side-problems))
  (assert (notify tech-department))
)

(defrule output-non-urgent-app-technical-failure-recommendation
  (service-type app)
  (problem-category technical-failure)
  (customer-urgency no)
  =>
  (printout t crlf "Your bug will be reported to tech department. Thank you for your feedback." crlf)
  (assert (recommendation report-average-app-bug-to-tech-department))
  (assert (notify tech-department))
)

```

Рисунок 23 – Предоставление рекомендации.

```

(defrule output-app-provide-info-recommendation
  (service-type app)
  (problem-category provide-info)
  (or (customer-urgency yes)
      (customer-urgency no))
  =>
  (printout t crlf "User guide is sent on your email" crlf)
  (assert (recommendation send-user-guide))
)

(defrule output-app-non-trivial-problem-recommendation
  (service-type app)
  (problem-category other)
  (or (customer-urgency yes)
      (customer-urgency no))
  =>
  (printout t crlf "You will be connected with our contact center to discuss a problem with your app" crlf)
  (assert (recommendation connect-with-contact-center-about-app-problem))
  (assert (notify contact-center))
)

```

Рисунок 24 – Предоставление рекомендации.

Таким образом общее число правил входящих в состав разработанной экспертной системы составляет 29.

3. Тестирование разработанной ЭС.

В таблице 2 представлено несколько основных веток возможного поведения программы с вводимыми данными и выводимыми рекомендациями системы.

Таблица 2. Основные ветки дерева решений разработанной ЭС.

Ответы на вопросы подряд	Рекомендация
System supports only card, contribution or app choices Enter a service where you face the problem: Card System supports only security-threat, technical-failure or provide-info choices. If nothing suits your case, you may choose option other. What type of problem happened with your card? technical-fail Is your problem critical? (yes or no) no	Your card will be re-released in seven days (recommendation non-urgent-card-release)
System supports only card, contribution or app choices Enter a service where you face the problem: Contribution System supports only security-threat, technical-failure or provide-info choices. If nothing suits your case, you may choose option other. What type of problem happened with your contribution? security-threat Is your problem critical? (yes or no) no	Your contribution is blocked by standart procedure. All recent operations will be inspected during till tomorrow. Technical support will call you for the next our (recommendation block-contribution-suspicious-transactions-inspection) (notify security-department) (notify tech-support)
System supports only card, contribution or app choices Enter a service where you face the problem: App System supports only security-threat, technical-failure or provide-info choices. If nothing suits your case, you may choose option other. What type of problem happened with your app? provide-info	User guide is sent on your email (recommendation send-user-guide)
System supports only card, contribution or app choices Enter a service where you face the problem: app System supports only security-threat,	Server-side problems. Retry your attempt in 30 minutes (recommendation report-about-server-side-problems) (notify tech-department)

technical-failure or provide-info choices. If nothing suits your case, you may choose option other. What type of problem happened with your app? technical-failure Is your problem critical? (yes or no) yes	
---	--

Далее представлены скриншоты преведенных в таблице ветвей дерева решений

CLIPS> (run)

```
System supports only card, contribution or app choices
Enter a service where you face the problem:
asd
```

```
System supports only card, contribution or app choices
Enter a service where you face the problem:
dd
```

```
System supports only card, contribution or app choices
Enter a service where you face the problem:
```

```
System supports only card, contribution or app choices
Enter a service where you face the problem:
card
==> f-1      (service-type card)
```

```
System supports only security-threat, technical-failure or provide-info choices.
If nothing suits your case, you may choose option other.
What type of problem happened with your card?
technical-failure
==> f-2      (problem-category technical-failure)
```

```
Is your problem critical? (yes or no)
```

```
Your card will be re-released in seven days
==> f-4      (recommendation non-urgent-card-re-release)
==> f-5      (notify tech-department)
CLIPS> █
```

Рисунок 25. Ветка дерева решений 1. С тестом валидации типа сервиса.

```

CLIPS> (run)

System supports only card, contribution or app choices
Enter a service where you face the problem:
contribution

System supports only card, contribution or app choices
Enter a service where you face the problem:
contribution

System supports only card, contribution or app choices
Enter a service where you face the problem:

System supports only card, contribution or app choices
Enter a service where you face the problem:
contribution
==> f-1      (service-type contribution)

System supports only security-threat, technical-failure or provide-info choices.
If nothing suits your case, you may choose option other.
What type of problem happened with your contribution?
security-threat
==> f-2      (problem-category security-threat)

Is your problem critical? (yes or no)
■

Your contribution is blocked by standart procedure. All recent operations will be inspected during till tomorrow.
==> f-4      (recommendation block-contribution-suspicious-transactions-inspection)
==> f-5      (notify security-department)
==> f-6      (notify tech-support)
CLIPS> ■

```

Рисунок 26. Ветка дерева решений 2. С тестом валидации типа сервиса.

```

CLIPS> (run)

System supports only card, contribution or app choices
Enter a service where you face the problem:
app
==> f-1      (service-type app)

System supports only security-threat, technical-failure or provide-info choices.
If nothing suits your case, you may choose option other.
What type of problem happened with your app?
provide onfo

System supports only security-threat, technical-failure or provide-info choices.
If nothing suits your case, you may choose option other.
What type of problem happened with your app?
provide-info
==> f-2      (problem-category provide-info)
==> f-3      (customer-urgency no)

User guide is sent on your email
==> f-4      (recommendation send-user-guide)
CLIPS>

```

Рисунок 27. Ветка дерева решений 3. С тестом валидации типа проблемы.

```

CLIPS> (run)

System supports only card, contribution or app choices
Enter a service where you face the problem:
app
==> f-1      (service-type app)

System supports only security-threat, technical-failure or provide-info choices.
If nothing suits your case, you may choose option other.
What type of problem happened with your app?
technical-failure
==> f-2      (problem-category technical-failure)

Is your problem critical? (yes or no)
yes
==> f-3      (customer-urgency yes)

Server-side problems. Retry your attempt in 30 minutes
==> f-4      (recommendation report-about-server-side-problems)
==> f-5      (notify tech-department)
CLIPS>

```

Рисунок 28. Ветка дерева решений 4.

Выводы.

В ходе выполнения лабораторной работы изучены базовые команды и конструкции языка CLIPS. Была создана ЭС, которая на основе пользовательского ввода выдает рекомендацию по действиям менеджеру в банковской области. Система динамически адаптирует процесс опроса в зависимости от предыдущих ответов. В ходе работы столкнулись с проблемой разного количества исходных данных, для разных рекомендаций ЭС, для решения проблемы было использовано правило для задания дефолтного значения. Также столкнулись с проблемой очередности отработки правила дефолтного значения, для решения этой проблемы был использован подход с понижением приоритета правила.

ПРИЛОЖЕНИЕ А

НАЗВАНИЕ ПРИЛОЖЕНИЯ

Файл task1.cpl.

```
(assert (car fit) (driver wait) (transaction success))

(deffacts parking "Pay parking system"
  (car fit)
  (driver wait)
  (transaction success)
)

(defrule R1
  (car fit)
  (driver wait)
  =>
  (printout t crlf "An empty parking spot for waiting driver
exists" crlf)
  (assert (spot available))
)
(defrule R2
  (driver wait)
  (transaction success)
  =>
  (printout t crlf "Transaction successfull: a waiting driver
paid money for parking" crlf)
  (assert (payment received))
)
(defrule R3
  (spot available)
  (payment received)
  =>
  (printout t crlf "Driver can park the car in the assigned
spot" crlf)
  (assert (parking granted))
)
```

Файл rulebase.cpl.

```
(deffunction validate-input (?input-text ?valid-options)
  (while TRUE
    (printout t crlf ?input-text crlf)
    (bind ?input (read))
    (if (member$ ?input ?valid-options)
      then (return ?input)
    )
  )
)

(defrule get-service-type
  =>
  (bind ?service-type (validate-input
    "System supports only card, contribution or app choices
Enter a service where you face the problem: "
```

```

        (create$ card contribution app)
    ))
    (assert (service-type ?service-type))
)

(defrule get-card-problem
  (service-type card)
  =>
  (bind ?problem-category (validate-input
    "System supports only security-threat, technical-failure or
provide-info choices.
If nothing suits your case, you may choose option other.
What type of problem happened with your card?"
    (create$ security-threat technical-failure provide-info
other)
  ))
  (assert (problem-category ?problem-category))
)

(defrule get-contribution-problem
  (service-type contribution)
  =>
  (bind ?problem-category (validate-input
    "System supports only security-threat, technical-failure or
provide-info choices.
If nothing suits your case, you may choose option other.
What type of problem happened with your contribution?"
    (create$ security-threat technical-failure provide-info
other)
  ))
  (assert (problem-category ?problem-category))
)

(defrule get-app-problem
  (service-type app)
  =>
  (bind ?problem-category (validate-input
    "System supports only security-threat, technical-failure or
provide-info choices.
If nothing suits your case, you may choose option other.
What type of problem happened with your app?"
    (create$ security-threat technical-failure provide-info
other)
  ))
  (assert (problem-category ?problem-category))
)

(defrule get-technical-failure-card-problem-urgency
  (service-type card)
  (problem-category technical-failure)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

```

```

)

(defrule get-non-trivial-card-problem-urgency
  (service-type card)
  (problem-category other)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-security-threat-contribution-problem-urgency
  (service-type contribution)
  (problem-category security-threat)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-technical-failure-contribution-problem-urgency
  (service-type contribution)
  (problem-category technical-failure)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-non-trivial-contribution-problem-urgency
  (service-type contribution)
  (problem-category other)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"
    (create$ yes no)
  ))
  (assert (customer-urgency ?customer-urgency))
)

(defrule get-technical-failure-app-problem-urgency
  (service-type app)
  (problem-category technical-failure)
  (not (recommendation ?))
  =>
  (bind ?customer-urgency (validate-input
    "Is your problem critical? (yes or no)"

```



```

        (create$ yes no)
    ))
    (assert (customer-urgency ?customer-urgency))
)

(defrule set-default-low-urgency-if-not-provided
  (declare (salience -100))
  (service-type ?)
  (problem-category ?)
  (not (customer-urgency ?))
=>
  (assert (customer-urgency no))
)

(defrule output-card-security-threat-recommendation
  (service-type card)
  (problem-category security-threat)
  (or (customer-urgency yes)
      (customer-urgency no))
=>
  (printout t crlf "Your card will be blocked. Info sent to
security department" crlf)
  (assert (recommendation block-card))
  (assert (notify security-department))
)

(defrule output-card-urgent-technical-failure-recommendation
  (service-type card)
  (problem-category technical-failure)
  (customer-urgency yes)
=>
  (printout t crlf "Your card will be re-released in two days"
crlf)
  (assert (recommendation urgent-card-re-release))
  (assert (notify tech-department))
)

(defrule output-card-non-urgent-technical-failure-recommendation
  (service-type card)
  (problem-category technical-failure)
  (customer-urgency no)
=>
  (printout t crlf "Your card will be re-released in seven days"
crlf)
  (assert (recommendation non-urgent-card-re-release))
  (assert (notify tech-department))
)

(defrule output-card-provide-info-recommendation
  (service-type card)
  (problem-category provide-info)
  (or (customer-urgency yes)
      (customer-urgency no))
=>
  (printout t crlf "Application to receive information about your
card is transmitted for consideration" crlf)
  (assert (recommendation transmit-card-info-application))
)

```

```

        (assert (notify tech-support))
    )

    (defrule output-card-non-trivial-urgent-problem-recommendation
        (service-type card)
        (problem-category other)
        (customer-urgency yes)
        =>
        (printout t crlf "You will be connected with our contact center
in a few minutes to discuss a problem with your card" crlf)
        (assert (recommendation connect-with-contact-center-about-
urgent-card-problem))
        (assert (notify contact-center))
    )

    (defrule output-card-non-trivial-non-urgent-problem-recommendation
        (service-type card)
        (problem-category other)
        (customer-urgency no)
        =>
        (printout t crlf "Our first available operator will recall you
to discuss problem with your card" crlf)
        (assert (recommendation connect-with-contact-center-about-non-
urgent-card-problem))
        (assert (notify contact-center))
    )

    (defrule output-urgent-contribution-security-threat-recommendation
        (service-type contribution)
        (problem-category security-threat)
        (customer-urgency yes)
        =>
        (printout t crlf "Your contribution is blocked. All recent
operations will be inspected during the next few hours" crlf)
        (assert (recommendation block-contribution-with-fraud-
inspection))
        (assert (notify security-department))
    )

    (defrule output-non-urgent-contribution-security-threat-
recommendation
        (service-type contribution)
        (problem-category security-threat)
        (customer-urgency no)
        =>
        (printout t crlf "Your contribution is blocked by standart
procedure. All recent operations will be inspected during till tomorrow.
Technical support will call you for the next hour"
crlf)
        (assert (recommendation block-contribution-suspicious-
transactions-inspection))
        (assert (notify security-department))
        (assert (notify tech-support))
    )

    (defrule output-urgent-contribution-technical-failure-
recommendation

```

```

        (service-type contribution)
        (problem-category technical-failure)
        (customer-urgency yes)
    =>
        (printout t crlf "Your contribution will be fixed in an hour by
tech department." crlf)
        (assert (recommendation transmit-critical-bug-to-tech-
department))
        (assert (notify tech-department))
    )

    (defrule output-non-urgent-contribution-technical-failure-
recommendation
        (service-type contribution)
        (problem-category technical-failure)
        (customer-urgency no)
    =>
        (printout t crlf "Your contribution will be fixed till tomorrow
by tech department." crlf)
        (assert (recommendation transmit-average-bug-to-tech-
department))
        (assert (notify tech-department))
    )

    (defrule output-contribution-provide-info-recommendation
        (service-type contribution)
        (problem-category provide-info)
        (or (customer-urgency yes)
            (customer-urgency no))
    =>
        (printout t crlf "Application to receive information about your
contribution is transmitted for consideration" crlf)
        (assert (recommendation transmit-contribution-info-
application))
    )

    (defrule output-contribution-non-trivial-urgent-problem-
recommendation
        (service-type contribution)
        (problem-category other)
        (customer-urgency yes)
    =>
        (printout t crlf "You will be connected with our contact center
in a few minutes to discuss a problem with your contribution" crlf)
        (assert (recommendation connect-with-contact-center-about-
urgent-contribution-problem))
        (assert (notify contact-center))
    )

    (defrule output-contribution-non-trivial-non-urgent-problem-
recommendation
        (service-type contribution)
        (problem-category other)
        (customer-urgency no)
    =>
        (printout t crlf "Our first available operator will recall you
to discuss problem with your contribution" crlf)

```

```

        (assert (recommendation connect-with-contact-center-about-non-
urgent-contribution-problem))
        (assert (notify contact-center))
    )

    (defrule output-app-security-threat-recommendation
        (service-type app)
        (problem-category security-threat)
        (or (customer-urgency yes)
            (customer-urgency no))
        =>
        (printout t crlf "You account is temporarily blocked.
Information sent to security department" crlf)
        (assert (recommendation block-account))
        (assert (notify security-department))
    )

    (defrule output-urgent-app-technical-failure-recommendation
        (service-type app)
        (problem-category technical-failure)
        (customer-urgency yes)
        =>
        (printout t crlf "Server-side problems. Retry your attempt in
30 minutes" crlf)
        (assert (recommendation report-about-server-side-problems))
        (assert (notify tech-department))
    )

    (defrule output-non-urgent-app-technical-failure-recommendation
        (service-type app)
        (problem-category technical-failure)
        (customer-urgency no)
        =>
        (printout t crlf "Your bug will be reported to tech department.
Thank you for your feedback." crlf)
        (assert (recommendation report-average-app-bug-to-tech-
department))
        (assert (notify tech-department))
    )

    (defrule output-app-provide-info-recommendation
        (service-type app)
        (problem-category provide-info)
        (or (customer-urgency yes)
            (customer-urgency no))
        =>
        (printout t crlf "User guide is sent on your email" crlf)
        (assert (recommendation send-user-guide))
    )

    (defrule output-app-non-trivial-problem-recommendation
        (service-type app)
        (problem-category other)
        (or (customer-urgency yes)
            (customer-urgency no))
        =>

```

```
        (printout t crlf "You will be connected with our contact center  
to discuss a problem with your app" crlf)  
        (assert (recommendation connect-with-contact-center-about-app-  
problem))  
        (assert (notify contact-center))  
    )
```